Project 2: Configure a Computer Network

Development of a download application;

Configuration and study of a computer network;

Application and Report made by

Deniz Lopes Gunes (up202208453) & António Pedro Alves Pais (up202305444)

Summary

We were able to develop the ftp client application, following a modular design, implementing the parsing of the hostname, connecting to a socket and retrieving files, while allowing for anonymous or authenticated transfers. We also were able to fully establish the network on the lab, to then connect to the lab's FTP server.

Introduction

The main objective of the lab was to understand how a network is configured and what is the structure required to have computers communicate with each other and the internet. As a side project we also developed an implementation of an FTP client that uses the TCP protocol to handle reconnections, retransmissions, among other network layer actions.

Table of Contents

Part 1 - FTP Client Implementation

Methodology

We followed a very modular system, having a separate file for each major task. We split the tasks into 4 major groups:

- URL Parser, which parses the input URL and retrieves hostname, login information and file path.
- Hostname Resolver, which resolves the IP address belonging to that hostname.
- FTP Control, which connects to the control socket, and handle the initial connection and disconnection.
- FTP Data, which handles the data transferring part of the application.

The main loop of the application calls function that perform simpler tasks, with the goal of transferring the file from the provided URL using the FTP protocol.

High-level overview

The following is a high-level overview of how the application works. It is important to note that this application only implements the FTP protocol, and thus, does not handle retransmissions or reconnections like the first project. These are handled by the TCP protocol which the application runs on.

The main loop of the application consists of firstly parsing the input URL. If the following is not valid it return immediately. The URL parser looks for user credentials. If it does not find any default to anonymous/anonymous for the USER/PASS arguments of the FTP protocol. It then looks for the first '/' everything behind is interpreted as the hostname, and everything after is interpreted as the file path.

Having the hostname and login credentials parsed, we move onto the next step. The application now resolves the hostname into an IP in binary which is then converted to a string with the IP segments. This resolution is handled by the following piece of code.

ftp_resolver.c

```c
int resolve_hostname(const char *hostname, char *out_ip, size_t out_ip_size) {
  struct hostent *he = gethostbyname(hostname);

  if (!he || !he->h_addr_list[0]) {
    fprintf(stderr, "DNS resolution failed for host: %s\n", hostname);
    return -1;
  }

  const char *ip = inet_ntop(AF_INET, he->h_addr_list[0], out_ip, out_ip_size);

  if (!ip) {
    perror("inet_ntop");
    return -1;
  }

  return 0;
}
```

Figure 1: Hostname resolver function

The application then connects a socket to the resolved IP, this will be the control socket, used to send commands and get replies from the server. This is handled by the following code snippet.

ftp_control.c

```c
int ftp_connect(const char *ip, int port) {
  int sockfd = socket(AF_INET, SOCK_STREAM, 0);
  if (sockfd < 0) {
    perror("socket");
    return -1;
  }

  struct sockaddr_in addr;
  memset(&addr, 0, sizeof(addr));
  addr.sin_family = AF_INET;
  addr.sin_port = htons(port);

  if (inet_pton(AF_INET, ip, &addr.sin_addr) != 1) {
    perror("inet_pton");
    close(sockfd);
    return -1;
  }

  if (connect(sockfd, (struct sockaddr *)&addr, sizeof(addr)) < 0) {
    perror("connect");
    close(sockfd);
    return -1;
  }

  return sockfd;
}
```

Figure 2: Control socket connection

An additional high-level function is defined in the `ftp_control.c` package, that sends a command and reads the response from the server. This function resorts to simpler functions, one reads the response from the server while parsing the response code, and the other is a simple wrapper to send a command with format strings. The read response also resorts to another function that reads a single line. This way we can handle multiple line responses. This package also exposes an additional function to parse the data connection's ip address from a server reply. This function is defined in the code snippet bellow.

ftp_control.c

```c
int ftp_parse_pasv(const char *reply, char *ip_out, size_t ip_len,
                   int *port_out) {
  const char *p = strchr(reply, '(');
  if (!p)
    return -1;
  p++;

  int h1, h2, h3, h4, p1, p2;

  if (sscanf(p, "%d,%d,%d,%d,%d,%d", &h1, &h2, &h3, &h4, &p1, &p2) != 6) {
    return -1;
  }

  snprintf(ip_out, ip_len, "%d.%d.%d.%d", h1, h2, h3, h4);
  *port_out = p1 * 256 + p2;

  return 0;
}
```

Figure 3: Parse IP from pasv response

For data, we have a separate package `ftp_data.c` that exposes a function to connect a new socket with the data connection's IP address.

At this point, the client strictly follows the FTP protocol specification, coordinating control and data connections according to the standard passive-mode workflow.

The client follows the FTP protocol specification using passive mode. After establishing the control connection, it sends the PASV command and parses the server's response to obtain the IP address and port for the data connection. The client then sends the RETR command with the requested file path over the control connection. Upon receiving a positive preliminary reply, it establishes the data connection and continuously reads from the data socket until no more bytes are available. After the transfer completes, the data connection is closed, the client waits for the final completion reply from the server, sends the QUIT command on the control connection, and then terminates while printing the download statistics.

Part 2 - Configuration and Study of a Network

Part 2 / Exp 1- Configure an IP Network

The goal of this first experiment was to configure the IP addresses of the two computers so that they could communicate between each other. For this we had to configure the E1 interfaces of both TUX3 and TUX4.

We did so by connecting the E1 interfaces of both TUXes to the Switch and running the following commands:

On TUX3: sudo ifconfig if_e1 172.16.60.1/24

On TUX4: sudo ifconfig if_e1 172.16.60.254/24

After this we cleared the ARP tables on TUX3 and ran the ping command to ping TUX4.

Analyzing the Wireshark logs, we find that the first two entries are ARP packets. They show that TUX3 does not previously know the MAC address associated with TUX4's IP, meaning, it does not have that information in its ARP table. Because of this, it sends a Broadcast packet to resolve the address.

```
15 23.825787794  TPLink_fd:9a:47      Broadcast           ARP         42 Who has 172.16.60.254? Tell 172.16.60.1
16 23.825841734  TPLink_c2:17:49      TPLink_fd:9a:47     ARP         60 172.16.60.254 is at ec:75:0c:c2:17:49
```

Figure 4: ARP packets viewed on the wireshark log

The subsequent logs are clear and simple to interpret, as they are just standard ICMP Echo Request and Echo Reply packets.

```
 21 24.826056941  172.16.60.1         172.16.60.254       ICMP       98 Echo (ping) request  id=0x0aeb, seq=2/512, ttl=64 (reply in 22)
 22 24.826118050  172.16.60.254       172.16.60.1         ICMP       98 Echo (ping) reply    id=0x0aeb, seq=2/512, ttl=64 (request in 21)
```

Figure 5: ICMP packets viewed on the wireshark logs

Questions:

What are the ARP packets and what are they used for?

ARP (Address Resolution Protocol) packets are Data Link Layer frames used to map a known IP address to an unknown Data Link Layer address (MAC Address). They are stored in the ARP table which we had to flush to properly send the broadcast packet. They are used when a host (in this case TUX3) wants to send an IP packet to another machine (in this case TUX4) on the same network but does not know the destination's physical address (MAC Address). Without the MAC address, the ethernet frame cannot be constructed.

What are the MAC and IP addresses of ARP packets and why?

There are two types of ARP packets, Request and Reply.

The request, or how we've been calling it broadcast packet, sends a packet with the source as the MAC address of the sender (TUX3), with destination ff:ff:ff:ff:ff:ff.

This packet is sent on the network for everyone, and it's payload is the sender and target IP's.

The reply sends a packet to the original broadcaster so that it can know the MAC address of the originally unknown host. The source MAC of this packet is the target's MAC in this case TUX4, and the destination is he original requester's MAC address (TUX3).

What packets does the ping command generate?

Because we flushed the ARP table, the ping command first generates the ARP broadcast packet and after that generates ICMP protocol packets.

What are the MAC and IP addresses of the ping packets?

The ping packets, sent by TUX3 who's IP is (172.16.60.1) have this same source IP and have the IP address of TUX4 as destination (172.16.60.254). The MAC addresses are similar in logic, in this specific case they are, for

the source the MAC address of TUX3 (8c:90:2d:fd:9a:47) and for the destination the MAC address of TUX4 (ec:75:0c:c2:17:49)

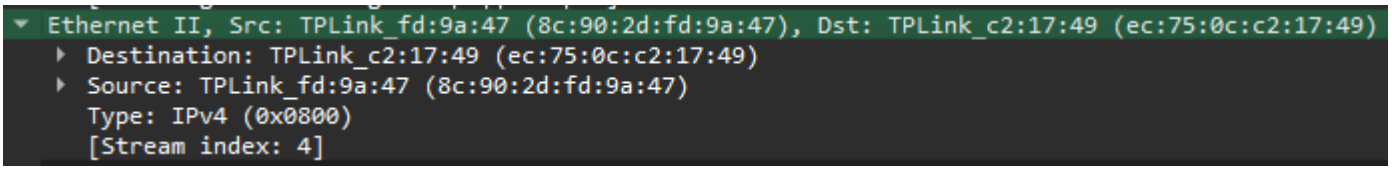

Figure 6: This figure shows the source and destination MAC addresses of a ICMP request packet generated by the ping command.

How to determine if a receiving Ethernet frame is ARP, IP, ICMP?

To detect if the packet is ARP or IP, we need to look at the EtherType field in the Ethernet header. If the value is 0x0806 then we are looking at an ARP packet. On the other hand, if the value is 0x0800, then we have an IP packet. ICMP is a type of IP packet, so if we see the EtherType is 0x0800 then we must look at the Protocol field. If this field's value is 1 then we have an ICMP packet.

How to determine the length of a receiving frame?

If we are dealing with an IP packet, the header has this information. The IP header's Total Length field specifies the size of the IP packet (header + payload). The Ethernet frame length must be determined from the Ethernet header and trailer.

What is the loopback interface and why is it important?

The loopback interface is an important channel that allows for messages to be sent and received in the same machine. Any message sent through this channel is immediately received by this same channel. It is important for example when developing web apps, for allowing processes to communicate between each other using sockets.

Part 2 / Exp 2 - Implement two bridges in a switch

The goal of this second experiment was to implement two bridges in the switch and analyse how network segmentation affects communication between different TUX machines.

Initially, it was necessary to repeat the network configuration from the first experiment and configure the IP address 172.16.61.1/24 for the TUX2. To create the bridges, it was necessary to configure the switch, which was done by connecting its console to the serial port of the TUX2.

In the configuration, bridges 60 and 61 were created (*/interface bridge add*), the ports to which the interfaces connected to each of the TUX were connected by default were removed (*/interface bridge port remove*), and these interfaces from TUX 3 and 4 were added to bridge60, and that of TUX2 to bridge61 (*/interface bridge port add*).

Questions:

How to configure bridgeY0?

The bridgeY0 is created in the switch to group the interfaces of TUX3 and TUX4 into the same LAN. To do this, we first add the bridge with the command (/interface bridge add name=bridgeY0).

After that, we removed the default port assignments (/interface bridge port remove [find interface=etherX]*) and added the ports connected to TUX3 and TUX4 (*/interface bridge port add bridge=bridgeY0 interface=etherX*).*

How many broadcast domains are there? How can you conclude it from the logs?

In this experiment there are two broadcast domains, which correspond to the two bridges created in the switch, bridgeY0 and bridgeY1. Each bridge isolates the broadcast traffic of the interfaces assigned to it, effectively creating separate LANs.

We can conclude this from the packet capture logs.

When a broadcast ping is sent from TUX3, which is in bridgeY0, only TUX4 receives and replies, since both belong to the same broadcast domain. TUX2, which is in bridgeY1, does not receive any of these packets.

When a broadcast ping is sent from TUX2, in bridgeY1, no other machine replies, because it is the only in that broadcast domain.

Part 2 / Exp 3 - Configure a Router in Linux

The goal of the third experiment was to transform TUX4 into a router to allow TUX3 and TUX2 to communicate between each other, being in different LANs. To achieve this, we configured if_e2 of TUX4 and added it to bridgeY1.

To achieve this we first setup TUX4 E2's ip address on the .61 network, running the following command.

- sudo ifconfig if_e2 172.16.61.253/24

On the switch we then had to add it to bridge 61 by running

- /interface bridge port remove [find interface=etherX]
- /interface bridge port add bridge=bridge61 interface=etherX

where X is the number of the port where we connected TUX4's E2.

After that, we enabled IP forwarding and disabled the option icmp_echo_ignore_broadcast to ensure that TUX4 could forward packets between the two networks, by running the two following commands:

- sudo sysctl -w net.ipv4.ip_forward=1
- sudo sysctl -w net.ipv4.icmp_echo_ignore_broadcasts=0

We reconfigured TUX3 and TUX2 to use TUX4's IP addresses as gateways, which we confirmed through the routing tables (route -n), by running the following commands:

- sudo route add -net 172.16.61.0/24 gw 172.16.60.254, on TUX3. This command tells TUX3 that to reach the subnet on 61.0, it needs to go through the only known interface (TUX4-E1) which is 172.16.60.254.
- sudo route add -net 172.16.60.0/24 gw 172.16.61.253, on TUX2. This command tells TUX2 that to reach the subnet on 60.0 it needs to go through the only known interface (TUX4-E2) which is 172.16.61.253 The logs confirmed that communication between TUX3 and TUX2, previously isolated, was now possible through TUX4 acting as a router.

Questions:

What routes are there in the tuxes? What are their meaning?

Each TUX machine had routes pointing to TUX4 as their gateway. TUX3 used the IP address of TUX4 in bridgeY0, while TUX2 used the IP address of TUX4 in bridgeY1. These routes meant that any packet destined for the other LAN was sent first to TUX4, which then forwarded it to the correct destination.

What information does an entry of the forwarding table contain?

A forwarding table entry contains the destination network or IP, the gateway to reach that destination, the interface through which the packet must be sent, and a metric that indicates the cost or priority of the route.

What ARP messages, and associated MAC addresses, are observed and why?

When TUX3 or TUX2 needed to send packets to the other LAN, they first had to discover the MAC address of their gateway, TUX4. ARP Requests were broadcasted to ask for the MAC address, and ARP Replies came from TUX4, providing its MAC. This explains why the MAC addresses observed in the frames belonged to TUX4, since

it was the router handling the traffic.

```
5.171709837   TPLink_fd:9a:47   TPLink_c2:17:49   ARP   42 172.16.60.1 is at 8c:90:2d:fd:9a:47
5.171684176   TPLink_c2:17:49   TPLink_fd:9a:47   ARP   60 Who has 172.16.60.1? Tell 172.16.60.254
```

Figure 7: This figure show the ARP packets when trying to ping TUX2 from TUX3

What ICMP packets are observed and why?

The ICMP packets observed were Echo Requests and Echo Replies generated by the ping command, used to test connectivity between TUX3 and TUX2. The router forwarded the ICMP packets across the two LANs, allowing successful communication that was not possible before TUX4 was configured as a router.

```
3.059744915   172.16.61.1   172.16.60.1   ICMP   98 Echo (ping) reply    id=0x0c78, seq=4/1024, ttl=63 (request in 10)
3.059437547   172.16.60.1   172.16.61.1   ICMP   98 Echo (ping) request  id=0x0c78, seq=4/1024, ttl=64 (reply in 11)
2.035757013   172.16.61.1   172.16.60.1   ICMP   98 Echo (ping) reply    id=0x0c78, seq=3/768, ttl=63 (request in 8)
2.035436584   172.16.60.1   172.16.61.1   ICMP   98 Echo (ping) request  id=0x0c78, seq=3/768, ttl=64 (reply in 9)
```

Figure 8: This figure shows the ICMP packets sent successfully between TUX2 and TUX3

What are the IP and MAC addresses associated to ICMP packets and why?

The IP addresses in the ICMP packets corresponded to the actual source and destination hosts (TUX3 and TUX2). However, the MAC addresses belonged to TUX4, because the router rewrote the Ethernet frame headers while forwarding the packets. This happens because routers preserve the IP layer information but replace the MAC addresses to ensure correct delivery at the data link layer.

```
Ethernet II, Src: TPLink_fd:9a:47 (8c:90:2d:fd:9a:47), Dst: TPLink_c2:17:49 (ec:75:0c:c2:17:49)
  Destination: TPLink_c2:17:49 (ec:75:0c:c2:17:49)
  Source: TPLink_fd:9a:47 (8c:90:2d:fd:9a:47)
```

Figure 9: This figure shows that an ICMP request packet sent from TUX3 has the MAC address of TUX4 as the destination

Part 2 / Exp 4 - Configure a Commercial Router and Implement NAT

The objective of this experiment is to configure the commercial router with NAT and allow for communication between the networks we had previously created with the internet, outside our private network. For this to happen, we had to configure the commercial router in a way that it was connected to the private network too.

To achieve this, after running all the previous experiments commands, we had to first reset the router itself. We accessed the router by switching the cable connect to TUX3 S0 from the switch to the router and using GMTK Term. We ran the following commands on the router:

/system reset-configuration, to reset the router's configuration

/ip address add address=172.16.1.61/24 interface=ether1, this command attributes an IP address to the outside interface of the router, which connects to the internet, and the lab's network

/ip address add address=172.16.61.254/24 interface=ether2, this command attributes an IP address to the inside interface of the router, which connects to our internal private network

/ip route add dst-address=172.16.60.0/24 gateway=172.16.61.253, this commands creates a route on the router that specifies how to reach the 60.0 subnet, which is through E2 of TUX4.

Now we had to add the internal interface of the router to bridge 61, we did this by running the following command on the switch's terminal:

/interface bridge port remove [find interface=etherRC_ETHER2]

/interface bridge port add bridge=bridge61 interface=etherRC_ETHER2

Where RC_ETHER2 is the port to which the routers Ether2 is connected to.

Finally we had to add the routes on every TUX to be able to communicate with the outside network. For TUX3 to reach the outside network we have to go through the router and to reach the router we have to first go through our only know interface (TUX4-E1). To achieve this we add the route on TUX3 with the following command:

sudo route add -net 172.16.1.0/24 gw 172.16.60.254, this commands tell TUX3 that to reach the outside network it needs to first go through TUX4 and then assumes that TUX4 knows how to reach the outside network/

To achieve this on TUX4 we add this route, specifying that to reach the outside network we have to go through the router, with the following command:

sudo route add -net 172.16.1.0/24 gw 172.16.61.254

Finally, on TUX2 we have two options, we can either route the requests first through TUX4 and then through the commercial router or directly through the Router, because they all have an interface on the same network. For this experiment we were told to send the requests directly to the Router, although we also tested with the other configuration. We achieve this by adding a route on TUX2 with the following command.

sudo route add -net 172.16.1.0/24 gw 172.16.61.254

The commercial router comes with the NAT table enabled by default so with this configuration we were able to ping the FTP server from any of the TUX2, TUX3 or TUX4.

Questions:

How to configure a static route in a commercial router?

To configure a static route, we must define the destination network (the subnet we want to reach) and the gateway (the next device IP address to send the traffic to).

What does NAT do?

The NAT or Network Address Translation allows devices on a private network, like the TUXes, to communicate with a public network while only exposing a single IP address. It stores on a table the internal ip of the request and then redirects the response to the requester.

What happens when tuxY3 pings the FTP server with the NAT disabled? Why?

The ping will fail. TUX3 knows how to reach the FTP server through the static routes we defined, namely the route to TUX4 which then routes to the Router which routes to the FTP server. The problem is that when the FTP server sends a response, when the NAT is disabled, the Router has no way to know who to route the response to and thus the packet is discarded so the request times out.

Part 2 / Exp 5 – DNS

This experiment was quite simpler than the others as it only involved configuring the Domain Name System (DNS) settings on our hosts to allow them to resolve human-readable domain names into IP addresses. Previously, we could only communicate using raw IP addresses. Now, by adding a DNS server, we enabled the use of hostnames.

To achieve this, we had to define the nameserver on every TUX (TUX2, TUX3, and TUX4). We did this by editing the system's resolver configuration file. We accessed each host and ran the following:

- sudo echo "nameserver 10.227.20.3" > /etc/resolv.conf

This command adds the laboratory DNS server (10.227.20.3) to the configuration file, telling the operating system to send name resolution requests to that specific IP address.

Questions:

How to configure the DNS service in a host?

In a Linux environment, the DNS is configured by editing the /etc/resolv.conf file.

What packets are exchanged by DNS and what information is transported

When a host needs to resolve a name, two main packets are exchanged using the UDP protocol on port 53. The first packet is the Query, sent from the Host to the DNS Server. It's payload is the hostname of the request and the record type that we are requesting, in this case Type A for IPv4. The second type is the Response which is sent from the DNS server back to the host. It's payload is the original query and the answer which matches the hostname to its corresponding IP address, that can then be used by the original host to execute the ping command in this case.

Part 2 / Exp 6 - TCP connections

For this experiment, we must first acknowledge some factors that took place. First, there were many problems regarding the university's FTP server. Thus, we were never able to experiment with two file downloads at the same time. Because of this we do not have wireshark logs to confirm our hypothesis. With that being said, we will still try to explain our though process regarding the congestion questions, but knowing that in practice it might have happened differently. This experiment was the cumulation of the previous experiments. We first built scripts for the three TUXes and for the Switch and Router. We ran each script in it's respective machine.

The scripts were as follows:

tux2.sh

```
#!/bin/bash
echo "Configuring Tux2 (Subnet 61)..."

# IP Configuration
sudo ifconfig if_e1 172.16.61.1/24

# Specific Route to Neighbor Subnet (60.0)
# We keep this to speak directly to Tux4 for internal traffic
sudo route add -net 172.16.60.0/24 gw 172.16.61.253

# Default Gateway
sudo route add -net 172.16.1.0/24 gw 172.16.61.254

# DNS Configuration
echo "nameserver 10.227.20.3" | sudo tee /etc/resolv.conf

echo "Tux2 Ready."
```

Figure 10: bash script ran in TUX2

tux3.sh

```
#!/bin/bash
echo "Configuring Tux3 (Subnet 60)..."

# IP Configuration
sudo ifconfig if_e1 172.16.60.1/24

# Clean ARP
sudo ip neigh flush all

# Default Gateway
sudo route add -net 172.16.61.0/24 gw 172.16.60.254
sudo route add -net 172.16.1.0/24 gw 172.16.60.254

# DNS Configuration
echo "nameserver 10.227.20.3" | sudo tee /etc/resolv.conf

echo "Tux3 Ready."
```

Figure 12: bash script ran on TUX3

tux4.sh

```
#!/bin/bash
echo "Configuring Tux4 (Router)..."

# Configure e1 on Subnet 60 and e2 on Subnet 61
sudo ifconfig if_e1 172.16.60.254/24
sudo ifconfig if_e2 172.16.61.253/24

# Enable IP Forwarding
sudo sysctl -w net.ipv4.ip_forward=1
sudo sysctl -w net.ipv4.icmp_echo_ignore_broadcasts=0

# Default Gateway
sudo route add -net 172.16.1.0/24 gw 172.16.61.254

# DNS Configuration
echo "nameserver 10.227.20.3" | sudo tee /etc/resolv.conf
```

Figure 11: bash script ran in TUX4

switch_commands.sh

```
/system reset-configuration

/interface bridge add name=bridge60
/interface bridge add name=bridge61

/interface bridge port remove [find interface=etherTUX3_E1]
/interface bridge port add bridge=bridge60 interface=etherTUX3_E1

/interface bridge port remove [find interface=etherTUX4_E1]
/interface bridge port add bridge=bridge60 interface=etherTUX4_E1

/interface bridge port remove [find interface=etherTUX2_E1]
/interface bridge port add bridge=bridge61 interface=etherTUX2_E1

/interface bridge port remove [find interface=etherTUX4_E2]
/interface bridge port add bridge=bridge61 interface=etherTUX4_E2

/interface bridge port remove [find interface=etherRC_ETHER2]
/interface bridge port add bridge=bridge61 interface=etherRC_ETHER2

/interface bridge port print brief
```

Figure 13: script ran on the Switch, with the appropriate replacements

After

router_commands.sh

```
/system reset-configuration

/ip address add address=172.16.1.61/24 interface=ether1
/ip address add address=172.16.61.254/24 interface=ether2

/ip route add dst-address=172.16.60.0/24 gateway=172.16.61.253

/ip route print
```

running the scripts, we ran the FTP client on TUX3 requesting a file from a public ftp server. We successfully downloaded and saved the file.

Figure 14: script ran on the Router

Questions:

How many TCP connections are opened by your FTP application?

There is a total of 2 sockets opened by our FTP application, one initially for the control and one for the data transfer.

In what connection is transported the FTP control information?

The FTP control information (such as USER, PASS, RETR, QUIT) is transported in the control connection.

What are the phases of a TCP connection?

A TCP connection has three main phases:

- Establishment – via the three-way handshake.
- Data transfer – where segments carrying application data are exchanged.
- Termination – using FIN and ACK packets to close the connection gracefully.

How does the ARQ TCP mechanism work? What are the relevant TCP fields? What relevant information can be observed in the logs?

TCP uses Automatic Repeat reQuest (ARQ) to ensure reliable delivery. Each segment carries a Sequence Number to identify the position of data in the stream, and the receiver responds with Acknowledgment Numbers to confirm receipt. In case a segment is lost, the sender retransmits it either after a timeout or upon receiving three duplicate ACKs.

Relevant TCP fields:

- Sequence Number (32 bits): Identifies the first byte of data in the segment
- Acknowledgment Number (32 bits): Indicates the next expected byte
- ACK flag (1 bit): Validates that the ACK number is meaningful
- Window Size (16 bits): Controls flow control and prevents buffer overflow
- Checksum (16 bits): Detects corruption in the segment

In the logs, we can observe retransmissions, ACKs confirming receipt, and sliding window adjustments that guarantee ordered and reliable delivery.

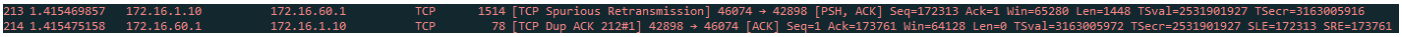

Figure 15: the figure shows a retranmission caught by wireshark in the file transfer proccess

How does the TCP congestion control mechanism work? What are the relevant fields. How did the throughput of the data connection evolve the time? Is it according to the TCP congestion control mechanism?

TCP congestion control uses these main algorithms: Slow Start, Congestion Avoidance, Fast Retransmit, and Fast Recovery.

Relevant fields:

- cwnd: Internal to the sender, controls how many bytes can be in flight
- ssthresh: Marks transition from exponential to linear growth
- Window Size: Advertised receiver window for flow control
- Sequence/Acknowledgment Numbers: Track data transmission and confirm receipt
- RTT: Calculated from timestamp options, affects timeout values
- Duplicate ACKs: Trigger Fast Retransmit without waiting for timeout

Although not present in the TCP header, variables such as cwnd and ssthresh are maintained by the sender's TCP stack and govern transmission behaviour.

The throughput grows rapidly at first, then more slowly, and eventually stabilizes. This matches the TCP congestion control: the rapid growth is Slow Start, the slower growth is Congestion Avoidance, and stabilization indicates the bandwidth limit was reached.

Is the throughput of a TCP data connections disturbed by the appearance of a second TCP connection? How?

Yes. When a second TCP connection starts during the transfer, the available bandwidth is shared between both connections.

Since TCP's congestion control ensures fairness, the throughput of the first connection decreases as the second one consumes part of the capacity.

Conclusion

The objectives of the project were fully developed. Due to constraints in the structure of the lab's FTP server we couldn't properly put into practice the TCP congestion control mechanism but from the theoretical classes and slides we were able to hypothesize on what would happen. The FTP client was also successfully implemented.

Annexes

A – FTP Client

The FTP Client can be accessed on the public repository:
https://github.com/denizlg24/rcom_project_2/tree/main/src

B – Configuration Scripts

The configuration scripts can be accessed on the public repository:
https://github.com/denizlg24/rcom_project_2/tree/main/bashscripts